Appendix A: What the room costs

Every argument in this book costs somebody a morning, and sooner or later the person who approves calendars is going to ask what that morning buys. This is the arithmetic I use when the question comes. It is not a business case, and the numbers in it are placeholders for numbers that belong to the team doing the counting.

Cost first, because that is the part nobody disputes. Five people in a room for seventy-five minutes is a little over six hours of engineering time. Add the preparation somebody does beforehand and the carry the Navigator does around it, and one gated spec runs to about seven engineer-hours. Two gated specs in an ordinary week is fourteen hours out of a five-person week of roughly two hundred, so the room takes something near seven percent of the team's capacity, and in the weeks where sessions cluster the way the template allows, it is closer to a tenth. One correction to that number first: if your team still runs refinement, this is a swap and not an addition. The session inherits refinement's content, and it can inherit its slot; part of the seven percent was already being spent, unexamined. That looks bad written down and it should.

One thing can soften it without shrinking it: where the agent has actually freed capacity, those hours come out of the dividend, not the principal. Whether it has is your team's number, not mine. Where it hasn't, the room is spending principal, and the seven percent is exactly what it looks like. Anyone approving the calendar does that multiplication before they read anything else, and a figure I decline to print is a figure they will assume I am hiding.

The other side of the ledger gets priced with my own wreckage rather than with a statistic I would have to caveat into uselessness. The split-PR round trip is the ordinary case, and it is the one the ratio below actually runs on: days rather than weeks, two people, call it twenty engineer-hours, and it also cost goodwill that never appeared on a ticket anywhere. JoustMania's player-history feature is the harder case, and I have to price it twice. Storage, pipelines, recording, replay, all of it built and none of it merged, because what got built was not what anybody had wanted. The building was the cheap half. An agent wrote most of it across a handful of evenings, and if this book is right about implementation, that number only keeps falling. What did not fall is the rest: several weeks of calendar time, a review request that sat open because nobody could

close it, the attention of everyone who looked at it and could not say why it felt wrong, and a token bill nobody counted. The forty engineer-hours I would have quoted a year ago are the wrong figure now, wrong in the direction that costs me the argument, so they are out of the ratio below rather than dressed up as still comparable.

Set that against seven hours a session and the room pays for itself when it catches one twenty-hour misbuild in three specs. That is the flattering version, and it is not the honest one. The question is not how often the team builds the wrong thing. It is how often the team builds the wrong thing in a way that a room full of people could have seen coming, and those are not the same number. Some answers only appear by building, which the Agora concedes, and this appendix has to concede it too. So discount the frequency by whatever fraction of the team's misbuilds were foreseeable at spec time. If half of them were, the bar moves to one in one and a half.

Which leaves a conclusion I would rather not have reached. Ordinary rework does not carry the room on its own, because an ordinary misbuild and the room itself get priced in the same currency, and a twenty-hour swing either way was never going to decide anything. What carries it is the expensive wrong turn, the JoustMania kind, the one that runs for weeks before anybody says it out loud and never reduces to a clean number of hours at all. Implementation got cheap. Discovering it was wrong didn't, and that gap is the column below that the arithmetic cannot count. How often those happen is not a number I can supply, and anybody who has shipped for a few years has a private answer that is worse than the one said in the retro.

Break-even is a threshold and not a target. A team that starts counting caught misbuilds in order to justify its sessions will find them, and the room has become a demo channel with a spreadsheet attached.

The uncounted column runs both ways. Against the room: calendar fragmentation is real, three sessions in a day genuinely wrecks people, and a session can produce a document nobody understood while it was written and nobody opens afterwards. For the room: the junior who learns the shape of the system by watching the argument, the onboarding that runs in weeks instead of months, the departure that does not take the system out of the door with it. Assigning those a number to win the column I want to win is the move the Oracle warns against, so they stay uncounted and this stays incomplete.

Testing it needs no platform. For one quarter, count gated specs, agent reruns, and tickets that come back as rework, then compare against the quarter before, which was badly recorded and contaminated by everything else that changed. A team that starts counting rework also starts finding it, so the old baseline is understated and any improvement will read smaller than it was. That error runs against the argument here, which is the direction I would rather have it run.

Which leaves the number that matters most, the one that would show all of this to be wrong. A team runs full sessions for a quarter, and the rework does not fall, and the forecasts do not tighten. If that happens, the book was wrong about that team, and the arithmetic above was decoration. I would rather hand over the figure that breaks the argument than the one that sells it.